

DIONÍSIO AGENT

Documento de Design de Software

Autor: Gabriel Negreiro Oliveira | Última atualização: 11 de junho de 2026

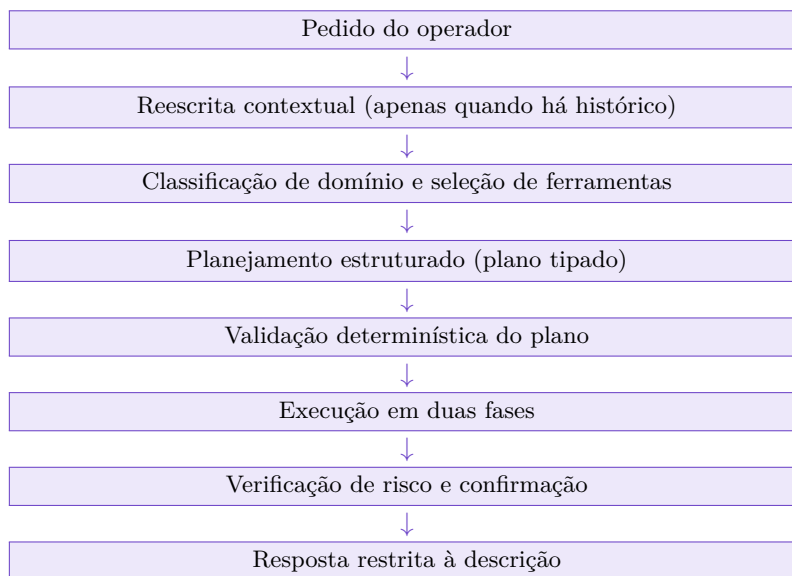
1 VISÃO GERAL E PRINCÍPIO ORIENTADOR

O Dionísio Agent é um assistente para um sistema de CRM de restaurantes. Ele recebe pedidos em linguagem natural, como consultas a clientes e reservas ou solicitações de ações operacionais, e os traduz em chamadas a uma API de domínio. A implementação é em Python, sem frameworks de orquestração de agentes, escolha que preserva o controle integral sobre os fluxos de execução, de segurança e de transação.

A premissa que organiza todo o projeto distingue dois tipos de componente. Chama-se *probabilístico* o modelo de linguagem, cujo resultado varia entre execuções e não pode ser garantido de antemão, e *determinístico* o código convencional, cujo comportamento é fixo e verificável. A tese de desenho é que a incerteza do modelo não deve ser eliminada, e sim isolada: o modelo propõe, o sistema valida e apenas o código executa. O modelo é usado em duas tarefas centrais, traduzir a linguagem natural em um plano estruturado e redigir a resposta final, enquanto a lógica que decide o que é verdade e o que será executado permanece em código testável. Esta versão acrescenta um segundo eixo à premissa: tarefas internas triviais, como a reescrita contextual, a classificação de apoio e a compressão de memória, são atendidas por um modelo rápido e barato, pois nelas um modelo maior não acrescenta qualidade, apenas latência e custo. As seções seguintes detalham as decisões que decorrem dessas premissas, as alternativas descartadas e o comportamento do sistema quanto a custo, confiabilidade e ações sensíveis.

2 DECISÕES DE ARQUITETURA

O processamento de um pedido percorre as etapas da figura.



A reescrita torna autônomas expressões dependentes do histórico, como “o telefone dele”, e é simplesmente pulada no primeiro turno, quando não há referência a resolver, o que elimina uma chamada ao modelo. A classificação seleciona quais conjuntos de ferramentas são relevantes, reduzindo o texto enviado ao planejador. O planejamento adota a estratégia de planejar antes de executar: o modelo produz o plano completo de uma só vez, como uma representação intermediária estruturada que o código interpreta, em contraste com abordagens reativas que decidem uma ação por vez; isso permite inspecionar e validar o plano inteiro antes que qualquer efeito ocorra.

O plano é composto por quatro tipos de nó: a chamada de ferramenta, que executa uma operação na API sob a verificação de risco; a computação, que aplica uma operação de dados determinística, chamada *data-op*, como filtrar, contar ou ordenar, sem o modelo e sem acesso à rede; a verificação, que avalia uma pré-condição e interrompe o plano se ela falha; e a iteração, que repete uma leitura para cada elemento de uma lista. A iteração é restrita a leituras e agora executa os elementos em paralelo, com teto configurável de tamanho e de conexões simultâneas, o que reduziu uma consulta dia a dia de cerca de noventa segundos para poucos segundos. Atribuir a filtragem e a contagem a um nó de computação, e não ao modelo, foi o que eliminou a

falha que motivou o redesenho original, a de respostas internamente inconsistentes nas quais o agente afirmava um número e listava outro.

Três propriedades completam a arquitetura. A resposta é restrita à descrição: a camada final apenas relata valores já calculados pelo código, recebe a data corrente para se situar no tempo e, quando um resultado é grande demais, recebe uma versão truncada com um marcador que preserva o total real, de modo que as quantidades citadas permanecem corretas e a resposta nunca chega vazia ao operador. A execução ocorre em duas fases: primeiro os passos reversíveis, que são leituras, computações e verificações; em seguida o conjunto de escritas é validado; somente então as escritas são executadas, por último. O contrato é tratado como fonte da verdade: o catálogo de ferramentas é confrontado automaticamente com a especificação OpenAPI por um teste de sincronização, e as formas reais das entidades, extraídas de respostas autênticas da API, são fornecidas ao planejador. Por fim, o texto enviado ao modelo é organizado por estabilidade: as instruções do planejador são congeladas na inicialização e todo conteúdo volátil, como datas, histórico e a consulta, fica na mensagem de usuário, separação que habilita o cache de prefixo descrito na seção 4.

3 ALTERNATIVAS CONSIDERADAS E DESCARTADAS

A renúncia a um framework de orquestração visou ao controle integral dos fluxos de segurança e de transação, ao custo de mais código próprio. Preferiu-se o planejamento antecipado à abordagem reativa porque o plano completo é inspecionável antes de agir, com a contrapartida de menor adaptabilidade durante a execução, atenuada por replanejamento. Para os predicados de filtro, adotou-se um formato declarativo restrito, composto por campo, operador e valor, em vez de código avaliado em ambiente isolado, por ser mais seguro e mais fácil de o modelo produzir corretamente. Na recuperação de ferramentas, quando o planejamento falha, optou-se por um único retorno ao catálogo completo, que é uma garantia, em vez de uma hierarquia de tentativas. Para a base de conhecimento, usou-se recuperação léxica por frequência de termos, conhecida como TF-IDF, em vez de busca semântica por vetores, que exigiria um provedor externo. Descartou-se um mecanismo completo de desfazimento, pois um efeito externo irreversível não admite reversão confiável; em seu lugar, o sistema relata sucesso parcial e oferece compensação quando existe operação inversa. Para reduzir o custo do planejador, considerou-se podar o catálogo e as instruções conforme a intenção detectada, mas preferiu-se o cache de prefixo do provedor, que preserva a cobertura integral do prompt sem arriscar perda de recall.

4 DESEMPENHO, CUSTO E ESCALABILIDADE

As camadas determinísticas têm custo proporcional ao tamanho do plano, e não ao número de ferramentas do catálogo, de modo que não se degradam quando o catálogo cresce. A verificação automática contra o contrato OpenAPI sustenta a manutenção do catálogo em escala. Ao passar de dezenas para centenas de operações, o limitante passa a ser o tamanho do texto enviado ao modelo; a evolução natural é substituir a classificação por palavras-chave por recuperação semântica, preservando o retorno ao catálogo completo como rede de segurança.

O custo por pedido é tratado como requisito de produto, não como curiosidade. Um módulo de telemetria registra, para cada chamada ao modelo, os tokens de entrada e saída, a latência e o custo informado pelo provedor, e o relatório de testes abre com esses agregados. Duas decisões reduzem esse custo. A primeira é a divisão em dois níveis de modelo, com o principal reservado ao planejamento e à redação. A segunda é o cache de prefixo: como as instruções do planejador são estáveis e o conteúdo volátil vem depois delas, o bloco estável é marcado para cache no provedor, e uma chamada repetida dentro da janela de validade paga cerca de um décimo do preço de entrada. Na medição ao vivo, a segunda chamada idêntica caiu de \$0,0689 para \$0,0058. Na bateria completa de testes, 56% dos tokens de entrada foram servidos do cache e o custo médio por pedido caiu de \$0,126 para \$0,088, uma redução de 31%, com latência média de 18,3 segundos. Esses números permitem estimar margem por operação: um operador com algumas dezenas de pedidos diários custa poucos dólares por dia, valor compatível com a precificação de um CRM por assento.

5 CONFIABILIDADE, ERROS E AMBIGUIDADE

A propriedade de não apresentar informação incorreta resulta da combinação entre a resposta restrita à descrição, as operações de dados determinísticas e a validação contra o contrato. Diante de erros, o sistema os classifica entre transitórios, que justificam nova tentativa, e definitivos, que não. Há uma exceção deliberada: escritas nunca são repetidas, pois uma falha de rede pode chegar depois de o servidor já ter aplicado a mutação e, sem chave de idempotência na API, repetir poderia duplicar o efeito. O sistema ainda exhibe ao operador mensagens limpas, mantendo o detalhe técnico em registro interno; repara o plano quando este não valida, mostrando ao modelo o erro exato; replaneja com o catálogo completo quando um passo falha; e mantém uma proteção geral em torno da execução, de forma que uma exceção inesperada nunca derruba a sessão.

Diante de ambiguidade, o agente prefere perguntar a adivinhar, e isso agora é mecanismo, não apenas intenção. O executor identifica os nós de busca cujo resultado alimenta uma escrita destrutiva; se mais de um registro corresponde à referência, por exemplo dois clientes chamados João antes de um cancelamento, o sistema interrompe, lista os candidatos e pede que o operador escolha, em vez de agir silenciosamente sobre o primeiro. Quando parte do pedido não corresponde a nenhuma ferramenta, o agente declara a limitação e realiza o que é possível, em vez de recusar o pedido inteiro.

A verificação empírica usa 31 cenários de ponta a ponta cobrindo leitura, ações destrutivas com confirmação e com recusa, casos de fronteira e perguntas sobre o produto. Cada cenário gera verificações independentes, entre elas a coerência do estado final, a ausência de informação interna vazada e, quando aplicável, um oráculo, isto é, um valor calculado por consulta direta à API que a resposta deve conter. A bateria é repetível: o único cenário que cria dados reais desfaz seu efeito ao final. No último ciclo, as 127 verificações foram aprovadas, e o mesmo resultado se manteve em execuções consecutivas com o código evoluindo entre elas; isso indica consistência sobre um conjunto curado, não prova de correção universal. Como indício de robustez, o mesmo código operou de um modelo de menor capacidade a um de maior sem alterações.

6 OPERAÇÕES SENSÍVEIS

Vale a regra direta: toda operação assinalada no contrato como `x-destructive: true`, como cancelar ou remarcar uma reserva, jamais é executada na primeira interação, ainda que o operador peça para “fazer direto”; ela exige sempre uma confirmação explícita subsequente. A execução em duas fases reforça essa proteção ao posicionar as escritas por último e ao validar o conjunto antes de qualquer mutação. Somam-se a isso a desambiguação descrita na seção anterior, que impede agir sobre a entidade errada quando há homônimos; a proibição de repetir escritas, que impede duplicar efeitos; e uma checagem que impede cancelar duas vezes o mesmo recurso na mesma sessão. Os parâmetros exibidos na confirmação são humanizados, convertendo, por exemplo, um instante em milissegundos para uma data legível, e enriquecidos com o nome da entidade afetada, para que o operador entenda exatamente o que aprova. Quando uma escrita falha após outra já concluída, o sistema relata o sucesso parcial e oferece a compensação correspondente, em vez de devolver uma falha que ocultaria o que ocorreu.

7 LIMITAÇÕES E EVOLUÇÃO

Algumas limitações conhecidas indicam a continuidade do trabalho. A latência média de 18 segundos é aceitável para um protótipo, mas alta para o balcão; o caminho é transmitir a resposta em fluxo contínuo e encurtar o planejamento de pedidos simples, e a interface já reduz a espera percebida exibindo o progresso de cada passo. O cache de prefixo cobre as instruções do planejador, e o catálogo por domínio é o próximo candidato. A desambiguação rastreia apenas a referência direta entre a busca e a escrita destrutiva, não cadeias intermediárias. A recuperação léxica da base de conhecimento é sensível à formulação da pergunta. A validação que antecede as escritas não cobre o caso em que uma escrita depende do resultado de outra, hoje tratado por relato e compensação, e a compensação é oferecida ao operador, mas não executada de forma autônoma. A memória de conversa vive no processo e não persiste entre sessões. Em todos esses pontos permanece a mesma tese que sustenta o projeto: o modelo propõe, o sistema valida e apenas o código executa.